



Telcoin-V3 Migration

Security Review

Cantina Managed review by:

Sujith Somraaj, Lead Security Researcher

Cryptara, Security Researcher

December 11, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Gas Optimization	4
3.1.1	Redundant pause state modifiers on <code>pause()</code> and <code>unpause()</code> functions	4
3.2	Informational	4
3.2.1	Migration End Time Not Enforced in Token Migration	4
3.2.2	Redundant <code>onlyMinter</code> modifier	4
3.2.3	Migration End Time Extension With Zero Liquidity	5
3.2.4	Use <code>Ownable2Step</code> instead of <code>Ownable</code>	5
3.2.5	Incorrect Event Emission in Migration End Time Update	6
3.2.6	Lack of granularity in <code>recoverERC20()</code> function for partial token recovery	6
3.2.7	Introduce upper bound validation in <code>updateMigrationEndTime()</code> function	6
3.2.8	Missing Constructor Parameter in Deployment Script	7
3.2.9	Constant name <code>TOTAL_SUPPLY</code> is misleading	7
3.2.10	Redundant <code>virtual</code> keyword on <code>interchainTokenService()</code> and <code>_spendAllowance()</code> functions	8
3.2.11	Function <code>tokenManagerCreate3Salt()</code> is redundant	8
3.2.12	Unused <code>originTEL</code> immutable variable	8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Telcoin creates low-cost, high-quality financial products for every mobile phone user in the world.

From Dec 4th to Dec 5th the Cantina team conducted a review of [token-migration](#) on commit hash [c5cad30a](#). The team identified a total of **13** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	1	1	0
Informational	12	11	1
Total	13	12	1

2.1 Scope

The security review had the following components in scope for [token-migration](#) on commit hash [c5cad30a](#):

```
src
├─ TelcoinV3.sol
└─ TokenMigration.sol
```

(Files changed in [PR 3](#))

3 Findings

3.1 Gas Optimization

3.1.1 Redundant pause state modifiers on `pause()` and `unpause()` functions

Severity: Gas Optimization

Context: `TelcoinV3.sol#L151-L155`

Description: The `pause()` and `unpause()` functions in `TelcoinV3.sol` contain redundant state validation modifiers that duplicate checks already performed in the underlying OpenZeppelin Pausable implementation.

The `whenNotPaused` modifier on `pause()` and `whenPaused` modifier on `unpause()` are redundant because:

1. OpenZeppelin's `Pausable._pause()` uses the `whenNotPaused` modifier.
2. OpenZeppelin's `Pausable._unpause()` uses the `whenPaused` modifier.

This results in duplicate validation logic and unnecessary gas consumption for users calling these functions, as the same state check is performed twice.

Recommendation: Remove the redundant modifiers from both functions.

Telcoin Association: Fixed in commit `7c97ca9b`.

Cantina Managed: Fix verified.

3.2 Informational

3.2.1 Migration End Time Not Enforced in Token Migration

Severity: Informational

Context: `TokenMigration.sol#L70-L93`

Description: The `TokenMigration` contract contains a critical design flaw where the `migrationEndTime` timestamp is not enforced in the `migrate` function. This variable is intended to define a migration window during which users can swap their old tokens for new `TelcoinV3` tokens. However, the current implementation only uses `migrationEndTime` as a gating mechanism for the owner's `withdrawRemainingTelcoinV3` function, preventing the owner from withdrawing unused tokens until the period expires.

The `migrate` function lacks any validation to ensure that migrations can only occur before `migrationEndTime`. As a result, users can continue migrating their tokens indefinitely, even after the intended migration period has ended. This breaks a fundamental invariant of the migration system that was designed to create a time-bound incentive mechanism for users to migrate promptly.

Furthermore, this undermines the stated purpose mentioned in the codebase comments about creating an intentional incentive mechanism through a migration window risk, where users who do not migrate within the timeframe would lose access to their tokens. Without enforcement of the deadline in the `migrate` function, this incentive mechanism is non-functional.

Recommendation: Implement a check in the `migrate` function to enforce the migration end time boundary. The function should revert if the current block timestamp is greater than or equal to `migrationEndTime`, ensuring that migrations can only occur during the intended window.

Telcoin Association: Fixed in commit `bc16c711`. `migrationEndTime` was renamed to `ownerAccessTimestamp` and the natspec documentation was updated in commit `bc16c71` to clarify the variable's intent.

Cantina Managed: Fix verified.

3.2.2 Redundant `onlyMinter` modifier

Severity: Informational

Context: [TelcoinV3.sol#L35-L39](#)

Description: The inherited `Minter.sol` contract already provides the `onlyRole(uint8(Roles.MINTER))` modifier through `RolesBase`. Creating a wrapper modifier (`onlyMinter`) adds unnecessary bytecode, increases gas costs, and duplicates functionality that already exists in the inherited role system.

Recommendation: Consider using the inherited `onlyRole(uint8(Roles.MINTER))` modifier directly on functions requiring minter access and remove `onlyMinter` modifier.

Telcoin Association: Fixed in commit [11262978](#).

Cantina Managed: Fix verified.

3.2.3 Migration End Time Extension With Zero Liquidity

Severity: Informational

Context: [TokenMigration.sol#L119](#)

Description: The `updateMigrationEndTime` function in the `TokenMigration` contract allows the owner to extend the migration deadline without validating whether the contract holds any `TelcoinV3` liquidity. In practice, if the contract has zero liquidity, users cannot successfully migrate their tokens since the `migrate` function will revert with `InsufficientContractBalance`. Therefore, extending the migration period when no liquidity exists serves no functional purpose unless the owner intends to replenish the contract with additional `TelcoinV3` tokens through cross-chain transfers.

The only legitimate scenario for extending `migrationEndTime` with zero liquidity is when the owner expects incoming liquidity from another chain and wants to proactively extend the deadline before those funds arrive. However, without any documentation, comments, or on-chain indicators of this intention, extending the deadline in a zero-liquidity state could mislead users into believing they still have time to migrate when in reality no tokens are available. This creates confusion and may cause users to delay their migration unnecessarily, only to discover later that the contract remains unfunded.

Recommendation: Consider implementing a liquidity check in the `updateMigrationEndTime` function that reverts if the remaining `TelcoinV3` balance is zero, unless there is a documented operational requirement for cross-chain liquidity management. Alternatively, enhance the `MigrationEndTimeUpdated` event to include the current liquidity balance at the time of the extension, providing transparency to users and off-chain monitoring systems about whether the deadline extension is meaningful or if they should wait for confirmed liquidity deposits before attempting to migrate.

Telcoin Association: Fixed in commit [eab7f90f](#).

Cantina Managed: Fix verified.

3.2.4 Use `Ownable2Step` instead of `Ownable`

Severity: Informational

Context: [TelcoinV3.sol#L15](#), [TokenMigration.sol#L14](#)

Description: Both `TelcoinV3.sol` and `TokenMigration` inherit from OpenZeppelin's `Ownable` contract, which allows single-step ownership transfers via `transferOwnership()`. This poses operational security risks:

1. Fat-finger errors: Accidentally transferring ownership to an incorrect address.
2. Irrecoverable ownership: No confirmation mechanism before ownership is transferred.

Given that both contracts have critical owner-only functions (minting, pausing, and withdrawing funds), an accidental ownership transfer could result in permanent loss of control over the protocol.

Recommendation: Replace `Ownable` with `Ownable2Step` in both contracts.

Telcoin Association: Fixed in commit [df79aac5](#).

Cantina Managed: Fix verified.

3.2.5 Incorrect Event Emission in Migration End Time Update

Severity: Informational

Context: `TokenMigration.sol#L123`

Description: The `updateMigrationEndTime` function in the `TokenMigration` contract contains a logic error in its event emission. The function updates the `migrationEndTime` state variable before emitting the `MigrationEndTimeUpdated` event, which causes the event to emit incorrect values. Specifically, the event is defined as `MigrationEndTimeUpdated(uint256 oldTime, uint256 newTime)` and is called with `emit MigrationEndTimeUpdated(migrationEndTime, _newTime)`. However, since `migrationEndTime` is already updated to `_newTime` on the previous line, the event will emit `MigrationEndTimeUpdated(_newTime, _newTime)` instead of the intended `MigrationEndTimeUpdated(oldValue, _newTime)`.

Recommendation: Cache the current value of `migrationEndTime` in a local variable before updating the state variable, and use this cached value when emitting the event or emit the event before the storage update.

Telcoin Association: Fixed in commit `2001e7d2`.

Cantina Managed: Fix verified.

3.2.6 Lack of granularity in `recoverERC20()` function for partial token recovery

Severity: Informational

Context: `TokenMigration.sol#L133`

Description: The `recoverERC20()` function always recovers the entire balance of accidentally sent tokens, lacking the ability to specify a recovery amount. Suppose multiple users accidentally send the same token type (e.g., User A sends 10 USDC and User B sends 20 USDC). In that case, the owner cannot recover tokens on an individual-per-user basis.

Instead, the owner must:

1. Withdraw all 30 USDC to an external account.
2. Manually distribute 10 USDC to User A and 20 USDC to User B off-chain.

The whole recovery process might become hard to automate and cumbersome.

Recommendation: Consider adding an amount parameter to enable partial token recovery, improving operational efficiency and user experience.

Telcoin Association: Acknowledged. This function intends to forward the contract's balance of (non-TelcoinV3) tokens to a safe multisig operated by our support department. From this point, it can be manually split between recipients for user recovery. As such, this finding is acknowledged.

Cantina Managed: Acknowledged.

3.2.7 Introduce upper bound validation in `updateMigrationEndTime()` function

Severity: Informational

Context: `TokenMigration.sol#L120`

Description: The `updateMigrationEndTime()` function allows the owner to extend the migration period but lacks upper-bound validation, allowing the migration end time to be set to an arbitrarily large future timestamp.

1. Owner intends to extend by 1 year (365 days = 31,536,000 seconds) but accidentally enters the value as a timestamp for year 2100+, effectively locking funds for decades.
2. Since the function only allows increases (`_newTime > migrationEndTime`), making accidentally large values impossible to correct.
3. The `withdrawRemainingTelcoinV3()` requires `block.timestamp >= migrationEndTime`, an erroneously high end time permanently locks remaining TelcoinV3 tokens in the contract.

Recommendation: Add a maximum extension limit to prevent accidentally locking funds long-term:

```

+ uint256 public constant MAX_EXTENSION_PERIOD = 365 days; // 1 year max extension

function updateMigrationEndTime(uint256 _newTime) external onlyOwner {
    if (_newTime <= migrationEndTime) revert InvalidEndTime(_newTime);
+   if (_newTime > block.timestamp + MAX_EXTENSION_PERIOD) {
+       revert InvalidEndTime(_newTime);
+   }

    uint256 oldTime = migrationEndTime;
    migrationEndTime = _newTime;
    emit MigrationEndTimeUpdated(oldTime, _newTime);
}

```

Telcoin Association: Fixed in commit [7571fd8b](#).

Cantina Managed: Fix verified.

3.2.8 Missing Constructor Parameter in Deployment Script

Severity: Informational

Context: [MigrationDeployment.s.sol#L40-L43](#)

Description: The deployment script in `MigrationDeployment.s.sol` contains a critical error where the `TokenMigration` contract deployment is missing a required constructor parameter. The `TokenMigration` contract's constructor expects four parameters: `_oldToken`, `_telcoinV3`, `_owner`, and `_migrationDuration`. However, the deployment script only encodes three parameters when constructing the bytecode for the `CREATE3` factory deployment.

The script constructs the migration bytecode as `abi.encode(telcoinV2, predictedTelcoinV3, owner)`, which only provides values for the old token address, the new TelcoinV3 address, and the owner address. The fourth parameter `_migrationDuration`, which determines how long the migration window remains open by calculating `migrationEndTime = block.timestamp + _migrationDuration`, is completely omitted from the deployment arguments.

This will cause the deployment transaction to fail during contract creation since the constructor will not receive the expected number of arguments. The ABI encoding will be malformed, and the `CREATE3` factory will be unable to successfully deploy the `TokenMigration` contract.

Recommendation: Add the `_migrationDuration` parameter to the deployment script's bytecode encoding. The corrected line should be `abi.encode(telcoinV2, predictedTelcoinV3, owner, migrationDuration)` where `migrationDuration` is either passed as a function parameter to the `run` function or defined as a constant within the script based on the desired migration window length.

Telcoin Association: Fixed in commit [2c9a3b73](#).

Cantina Managed: Fix verified.

3.2.9 Constant name `TOTAL_SUPPLY` is misleading

Severity: Informational

Context: [TelcoinV3.sol#L20](#)

Description: The `TOTAL_SUPPLY` constant is set to 100 billion tokens, but the contract owner can mint unlimited additional tokens at any time using the `mint()` function, making the name misleading. The constant suggests a hard supply cap exists only during initial migration allocation, not a protocol-wide maximum.

Recommendation: Rename the constant to reflect its purpose accurately:


```
- uint256 public constant TOTAL_SUPPLY = 100_000_000_000 * 10 ** 18; // 100B tokens with  
↳ 18 decimals  
+ uint256 public constant MIGRATION_SUPPLY_CAP = 100_000_000_000 * 10 ** 18; // 100B  
↳ tokens with 18 decimals
```

Telcoin Association: Fixed in commit [48368132](#).

Cantina Managed: Fix verified.

3.2.10 Redundant `virtual` keyword on `interchainTokenService()` and `_spendAllowance()` functions

Severity: Informational

Context: [TelcoinV3.sol#L126](#), [TelcoinV3.sol#L131](#)

Description: Both `interchainTokenService()` and `_spendAllowance()` functions are marked as `virtual`, allowing further inheritance and overriding. However, `TelcoinV3` is the final implementation contract and is not intended to be inherited.

Recommendation: Consider removing the `virtual` keyword from both functions.

Telcoin Association: Fixed in commit [3f621f07](#).

Cantina Managed: Fix verified.

3.2.11 Function `tokenManagerCreate3Salt()` is redundant

Severity: Informational

Context: [TelcoinV3.sol#L100-L105](#)

Description: The `tokenManagerCreate3Salt()` function is a redundant wrapper that simply returns the value of `interchainTokenId()`. This adds unnecessary bytecode and introduces an additional function call when deriving the `TokenManager` address.

The function serves purely as semantic aliasing with no additional logic or validation.

Recommendation: Remove `tokenManagerCreate3Salt()` and use `interchainTokenId()` directly. Additionally, update the documentation for `interchainTokenId()` to clarify its dual purpose.

Telcoin Association: Fixed in commit [3b0e039c](#).

Cantina Managed: Fix verified.

3.2.12 Unused `originTEL` immutable variable

Severity: Informational

Context: [TelcoinV3.sol#L28](#)

Description: The `originTEL` immutable variable is stored in the contract but never used.

Recommendation: Consider removing the `originTEL` variable and its constructor parameter.

Telcoin Association: Fixed in commit [2c9a3b73](#).

Cantina Managed: Fix verified.